

Infografía

Módulo 7 — Godot 4 • Sesión 5.5

IA de navegación (1): steering y decisión

UPB • Gestión I/2026

Plan de la clase

1. ¿Qué es "IA" en un juego 2D? — mover y decidir.
2. **Steering** — seek y arrive (escena 01).
3. **Detección** — ¿el enemigo ve al jugador? (línea de vista).
4. **Máquina de estados** — patrullar, perseguir, regresar (escena 02).
5. **El problema** — el enemigo se traba contra el muro.

Sesión 5: construimos el mundo con tiles. Hoy: alguien que se mueve solo por ese mundo.

La pregunta del día

El jugador lo movés vos.
El enemigo... ¿quién lo mueve?

*La "IA" de un juego 2D son dos cosas: **cómo se mueve** y **qué decide hacer**. Hoy las dos, sin pathfinding todavía.*

Parte 1 — steering: seek (escena 01)

La forma más simple de moverse hacia algo: cada frame, apuntá la velocidad al objetivo.

```
var hacia := player.global_position - global_position
velocity = hacia.normalized() * max_speed
move_and_slide()
```

*Esto es **seek**: "buscar". El enemigo siempre encara al jugador.*

Parte 1 — steering: arrive

Seek puro se pasa de largo y tiembla encima del objetivo. **Arrive**: cuando estás cerca, bajá la rapidez de a poco.

```
var dist := hacia.length()
var rapidez := max_speed
if dist < arrive_radius:
    rapidez = max_speed * (dist / arrive_radius) # frena al acercarse
velocity = velocity.move_toward(hacia.normalized() * rapidez, ACCEL * delta)
```

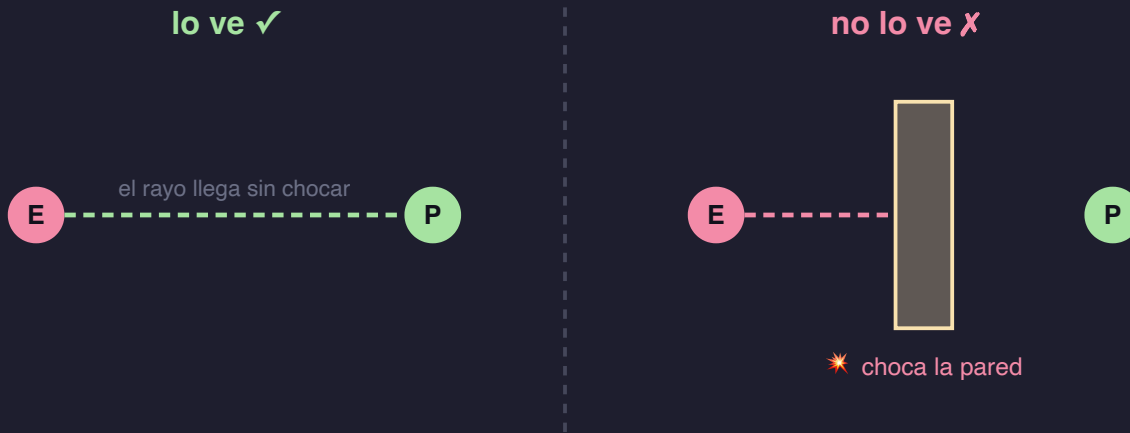
`move_toward` = *aceleración: no saltamos a la velocidad, llegamos de a poco.*

Parte 2 – detección: ¿me ve?

Perseguir todo el tiempo es aburrido (y tramposo). El enemigo debería **reaccionar** solo cuando te ve. "Ver" = dos condiciones:

- estás **cerca** (dentro de un rango de visión), y
- **no hay una pared** en el medio (línea de vista).

Línea de vista: un RayCast2D del enemigo al jugador



"ver" = está cerca Y `is_colliding() == false`

Parte 2 — línea de vista con RayCast2D

Un `RayCast2D` tira un rayo del enemigo al jugador y dice si chocó algo.

```
func _ve_al_jugador() -> bool:
    if global_position.distance_to(player.global_position) > vision:
        return false
    vista.target_position = vista.to_local(player.global_position)
    vista.force_raycast_update()
    return not vista.is_colliding()    # si chocó una pared, no lo ve
```

El rayo está en la capa `mundo`: solo lo cortan las paredes.

Parte 3 – decidir: máquina de estados (escena 02)

Igual que la sesión 4, pero el estado decide **comportamiento**, no animación.

El enemigo decide según su ESTADO

"ver" = está cerca Y no hay una pared en el medio (línea de vista)



Parte 3 — el **match** de siempre

```
enum Estado { PATRULLA, PERSIGUE, REGRESA }

func _physics_process(delta):
    match estado:
        Estado.PATRULLA: _patrulla(delta)    # ve al jugador -> PERSIGUE
        Estado.PERSIGUE: _persigue(delta)    # lo pierde      -> REGRESA
        Estado.REGRESA:  _regresa(delta)     # llega            -> PATRULLA
    move_and_slide()
```

Cada estado hace algo simple y dice cuándo cambiar. Eso es toda la "inteligencia".

El problema (y el gancho de la próxima)

PERSIGUE es steering en **línea recta**.
Funciona en campo abierto...
pero el enemigo **se traba contra el muro**.

*Conoce la **dirección** al jugador, no el **camino**. Eso lo resolvemos la próxima: pathfinding.*

Resumen

Moverse (steering):

- seek — encarar al objetivo
- arrive — frenar al llegar
- `move_toward` = aceleración

Decidir:

- detección por rango + **línea de vista** (RayCast2D)
- **máquina de estados** de comportamiento

La IA reactiva es rápida y barata. Su límite: no rodea obstáculos. → Sesión 5.6